Bài Giảng: 10 Thuật Toán Dự Báo Chuỗi Thời Gian Từ Cơ Bản Đến Deep Learning

Minh Triết

Ngày 12 tháng 3 năm 2026

Mục lục

1 Nhóm Thống Kê Kinh Điển (Classical Statistical Models) 3
1.1 Autoregressive (AR) . . . 3
1.2 Autoregressive Integrated Moving Average (ARIMA) . . . 3
1.3 Seasonal ARIMA (SARIMA) . . . 4
1.4 Exponential Smoothing (ES - Holt-Winters) . . . 4

2 Nhóm Học Máy & Cộng Gộp (Machine Learning & Additive) 5
2.1 XGBoost (Extreme Gradient Boosting) . . . 5
2.2 Prophet (Meta/Facebook) . . . 6

3 Nhóm Học Sâu (Deep Learning) 6
3.1 LSTM (Long Short-Term Memory) . . . 6
3.2 DeepAR (Amazon) . . . 6
3.3 N-BEATS (Neural Basis Expansion Analysis) . . . 7
3.4 Temporal Fusion Transformer (TFT - Google) . . . 7

4 CODE - KAGGLE 7
4.1 Đôi điều về cuộc thi Hedge Fund . . . 7
4.2 Thuật toán có thể lựa chọn ban đầu . . . 8
4.2.1 Mô hình hồi quy mạnh (Ưu tiên hàng đầu) . . . 8
4.2.2 Mô hình chuỗi thời gian thuần túy (Có thể kết hợp) . . . 8
4.2.3 Lưu ý đặc biệt từ dữ liệu quan sát được . . . 9
4.2.4 Đề xuất chiến lược thử nghiệm . . . 9
4.3 Thách thức . . . 10
4.4 Mô Hình Lỗi - Phân tích từ "Incorrect Modelling Notebook . . . 10
4.4.1 Lỗi 1: Chuẩn hóa target bằng thống kê toàn cục theo nhóm . . . 11
4.4.2 Lỗi 2: Tạo feature trung bình trên toàn bộ thời kỳ . . . 11
4.4.3 Lỗi 3: Tạo feature trễ (lag) bằng dữ liệu tương lai . . . 12
4.4.4 Tổng hợp so sánh . . . 12
4.4.5 Bài học rút ra . . . 12

5 Phân tích Pipeline "Competition v5 **13**
5.1 Tổng quan kiến trúc 13
5.2 Chi tiết các thành phần chính 13
5.2.1 Xử lý dữ liệu ban đầu (Cells 4-5) 13
5.2.2 Feature Engineering Horizon-Aware (Cell 6) 14
5.2.3 Purged Time-Series Cross-Validation (Cell 10) 15
5.2.4 Ensemble Training (Cell 13) 15
5.2.5 Hierarchical Reconciliation (Cell 14) 16
5.2.6 Post-Processing: 4 Luật Chống Nhiễu (Cell 15) 16
5.3 So sánh với Basic Pipeline 16
5.4 Điểm mạnh và điểm yếu 17
5.4.1 Điểm mạnh 17
5.4.2 Điểm yếu 17
5.5 Kết luận 17

Lời Mở Đầu

Việc chọn đúng "vũ khí"thuật toán là bước tiên quyết để chiến thắng trong các bài toán dự báo chuỗi thời gian, đặc biệt là với các yêu cầu khắt khe về *Sequential Prediction* (Dự báo tuần tự) để tránh *Look-ahead bias*. Tài liệu này hệ thống hóa 10 thuật toán từ nền tảng thống kê kinh điển đến các kiến trúc Deep Learning tối tân nhất.

1 Nhóm Thống Kê Kinh Điển (Classical Statistical Models)

Nhóm này coi chuỗi thời gian là một thực thể độc lập. Chúng chủ yếu giải quyết bài toán **Univariate** (chỉ dùng đúng một cột lịch sử giá trị để dự báo tương lai).

1.1 Autoregressive (AR)

- **Ý tưởng bài toán gốc:** Tương lai chỉ là cái bóng của quá khứ. Nếu ta biết giá trị của các ngày trước, ta có thể suy ra ngày hôm nay bằng một đường thẳng tuyến tính.
- **Hướng đi chính:** Dùng hồi quy tuyến tính (Linear Regression) áp dụng lên chính các độ trễ (lags) của dữ liệu.
- **Công thức cốt lõi:**

$$Y_t = c + \sum_{i=1}^{p} \phi_i Y_{t-i} + \epsilon_t$$

Trong đó:

- c: Hằng số (drift).
- ϕ_i: Hệ số tự hồi quy (trọng số của quá khứ).
- ϵ_t: Nhiễu trắng tại thời điểm t (kỳ vọng bằng 0, phương sai không đổi).
- **Điểm mạnh:** Cực kỳ đơn giản, tốc độ chạy tính bằng mili-giây, dễ dàng giải thích kết quả.
- **Điểm yếu:** Bất lực trước dữ liệu có xu hướng (trend) hoặc mùa vụ (seasonality). Bắt buộc chuỗi phải "dừng"(stationary).
- **Mục tiêu sử dụng:** Làm baseline (mức cơ sở) nhanh để so sánh, hoặc dùng cho các chuỗi dữ liệu nhiễu trắng/đã được xử lý tĩnh hoàn toàn.

1.2 Autoregressive Integrated Moving Average (ARIMA)

- **Ý tưởng bài toán gốc:** Dữ liệu thực tế hiếm khi "dừng". Hãy "triệt tiêu"xu hướng đó trước (lấy sai phân), sau đó mới áp dụng hồi quy.
- **Hướng đi chính:** Kết hợp học từ các giá trị trễ (AR), sai phân bậc d (I), và học từ các sai số dự báo trong quá khứ (MA).

- **Công thức cốt lõi (với toán tử trễ L):**

$$\left(1 - \sum_{i=1}^{p} \phi_i L^i\right)(1-L)^d Y_t = c + \left(1 + \sum_{j=1}^{q} \theta_j L^j\right)\epsilon_t$$

  Trong đó:

  - $(1-L)^d$: Sai phân bậc d.
  - θ_j: Hệ số trung bình trượt tác động lên nhiễu ϵ_{t-j}.
  - **Điểm mạnh:** Nền tảng toán học vững chắc, xử lý tốt chuỗi có xu hướng phi tuyến.
  - **Điểm yếu:** Không bắt được chu kỳ mùa vụ. Khó tinh chỉnh tự động hóa trên quy mô lớn.
  - **Mục tiêu sử dụng:** Các chỉ số kinh tế vĩ mô (GDP, lạm phát) hoặc giá cổ phiếu ngắn hạn không có tính mùa vụ.

1.3 Seasonal ARIMA (SARIMA)

- **Ý tưởng bài toán gốc:** Thêm một "bộ đếm thời gian"riêng biệt rà soát lại chu kỳ lặp lại vào mô hình ARIMA cơ sở.
- **Hướng đi chính:** Chồng thêm một lớp ARIMA thứ hai hoạt động song song trên các bước nhảy thời gian dài (seasonal lags).
- SARIMA$(p,d,q)(P,D,Q)_m$ mở rộng ARIMA bằng cách thêm các thành phần tương tự nhưng ở độ trễ mùa vụ m (ví dụ: $m = 12$ cho dữ liệu tháng).
- $$\Phi_P(L^m)\phi_p(L)(1-L^m)^D(1-L)^d Y_t = c + \Theta_Q(L^m)\theta_q(L)\epsilon_t$$
- Φ_P, Θ_Q: Đa thức AR và MA cho thành phần mùa vụ.
- Mô hình này bắt buộc chuỗi phải có tính chu kỳ cố định và rõ ràng.
- **Điểm mạnh:** Giải quyết triệt để các bài toán có chu kỳ lặp lại cố định.
- **Điểm yếu:** Rất nặng về tính toán nếu chu kỳ quá dài. Chỉ bắt được một loại mùa vụ duy nhất.
- **Mục tiêu sử dụng:** Dự báo doanh số bán lẻ hàng tháng, tiêu thụ điện năng.

1.4 Exponential Smoothing (ES - Holt-Winters)

- **Ý tưởng bài toán gốc:** Ký ức xa dần theo thời gian. Giá trị gần hiện tại quan trọng hơn quá khứ xa xôi.
- **Hướng đi chính:** Cập nhật các trạng thái liên tục bằng cách lấy trung bình có trọng số giảm dần theo hàm mũ.

- ES dự báo dựa trên các phương trình cập nhật trạng thái (State Space):

$$
\begin{aligned}
L_t &= \alpha(Y_t - S_{t-m}) + (1-\alpha)(L_{t-1} + T_{t-1}) \\
T_t &= \beta(L_t - L_{t-1}) + (1-\beta)T_{t-1} \\
S_t &= \gamma(Y_t - L_{t-1} - T_{t-1}) + (1-\gamma)S_{t-m} \\
\hat{Y}_{t+h} &= L_t + hT_t + S^{+}_{t-m+h_m}
\end{aligned}
$$

- L_t, T_t, S_t: Mức độ (Level), Xu hướng (Trend), và Mùa vụ (Seasonality) tại thời điểm t.
- $\alpha, \beta, \gamma \in [0, 1]$: Các siêu tham số làm phẳng (Smoothing parameters).
- **Điểm mạnh:** Nhanh gọn, mạnh mẽ, xử lý rất tốt dữ liệu có trend thay đổi từ từ qua thời gian. Khả năng ngoại suy (extrapolate) tốt.
- **Điểm yếu:** Giới hạn ở tính tuyến tính, không thể đưa thêm các biến ngoại sinh tự nhiên.
- **Mục tiêu sử dụng:** Quản lý hàng tồn kho, chuỗi cung ứng ngắn hạn.

2 Nhóm Học Máy & Cộng Gộp (Machine Learning & Additive)

Nhóm này cho phép nhồi nhét vô số thông tin bên ngoài (Multivariate) vào mô hình.

2.1 XGBoost (Extreme Gradient Boosting)

- **Ý tưởng bài toán gốc:** Trải phẳng thời gian thành các cột dữ liệu (Lag, Rolling) và dùng cây quyết định để học.
- **Hướng đi chính:** Tối ưu hóa hàm mục tiêu cộng gộp dựa trên Gradient Boosting.
- **Công thức phạt (Regularization):**

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

- **Điểm mạnh:** Độ chính xác cực cao, xử lý tốt dữ liệu khuyết, kết hợp hoàn hảo các đặc trưng tĩnh (code, category). Rất phù hợp với bộ dữ liệu quỹ phòng hộ có tính cấu trúc.
- **Điểm yếu:** Không thể ngoại suy xu hướng ra ngoài khoảng dữ liệu train. Dễ dính Look-ahead bias nếu Feature Engineering sai.
- **Mục tiêu sử dụng:** Các bài thi Kaggle phức tạp, hệ thống giao dịch định lượng.

2.2 Prophet (Meta/Facebook)

- **Ý tưởng bài toán gốc:** Coi chuỗi thời gian là sự kết hợp của Đường xu hướng + Đường chu kỳ + Ngày lễ.
- **Hướng đi chính:** Tiếp cận bằng mô hình cộng gộp (GAM) với các hàm toán học (Logistic trend, Fourier series).
- **Công thức cốt lõi:**
$$Y_t = g(t) + s(t) + h(t) + \epsilon_t$$
- **Điểm mạnh:** Tự động hóa cực tốt, chịu lỗi rác (outliers) giỏi, thiết lập ngày lễ dễ dàng.
- **Điểm yếu:** Bỏ qua tính tự hồi quy (Autoregressive) của dữ liệu.
- **Mục tiêu sử dụng:** Phân tích lưu lượng web, doanh số quy mô doanh nghiệp.

3 Nhóm Học Sâu (Deep Learning)

Được thiết kế để xử lý dữ liệu đa biến, học các biểu diễn ẩn phức tạp.

3.1 LSTM (Long Short-Term Memory)

- **Ý tưởng bài toán gốc:** Tạo cơ chế cho mạng Neural tự quyết định nên "nhớ"hay "quên"thông tin lịch sử.
- **Hướng đi chính:** Dùng cấu trúc cổng (Gates: Forget, Input, Output) kiểm soát luồng thông tin qua Cell state.
- **Điểm mạnh:** Khả năng học phi tuyến tính cực độ, nhớ được các phụ thuộc tầm xa.
- **Điểm yếu:** Chậm, khó tối ưu, là một hộp đen (Black box) hoàn toàn.
- **Mục tiêu sử dụng:** Dữ liệu tần số cao, IoT.

3.2 DeepAR (Amazon)

- **Ý tưởng bài toán gốc:** Dự báo một con số duy nhất là vô nghĩa, cần biết phân phối xác suất để quản trị rủi ro.
- **Hướng đi chính:** Dùng RNN xuất ra các tham số (ví dụ μ, σ) của một phân phối xác suất.
- **Điểm mạnh:** Probabilistic forecasting, giải quyết tốt bài toán Cold-start (sản phẩm/mã mới).
- **Điểm yếu:** Cấu trúc phức tạp, cần số lượng lớn chuỗi thời gian liên quan để huấn luyện.
- **Mục tiêu sử dụng:** Tối ưu hóa tồn kho bán lẻ (hàng vạn SKU), quản trị rủi ro tài chính.

3.3 N-BEATS (Neural Basis Expansion Analysis)

- **Ý tưởng bài toán gốc:** Tạo ra mạng Deep Learning có sức mạnh phi tuyến nhưng vẫn diễn giải được như mô hình thống kê.
- **Hướng đi chính:** Xếp chồng các khối MLP học phần dư (residuals), ép mạng tách riêng tín hiệu Trend và Seasonality.
- **Điểm mạnh:** Huấn luyện nhanh, trực quan hóa được thành phần chuỗi, hiệu suất cao.
- **Điểm yếu:** Tốn nhiều RAM nếu cửa sổ thời gian (lags) lớn.
- **Mục tiêu sử dụng:** Khi cần Deep Learning nhưng có yêu cầu khắt khe về tính minh bạch và giải thích được.

3.4 Temporal Fusion Transformer (TFT - Google)

- **Ý tưởng bài toán gốc:** Tích hợp thanh lịch 3 loại thông tin: Đặc trưng tĩnh, Quá khứ đã biết, Tương lai đã biết.
- **Hướng đi chính:** Dùng Multi-Head Attention nhận diện mốc thời gian quan trọng, kết hợp Gated Residual Network lọc biến.
- **Điểm mạnh:** Kiến trúc State-of-the-Art cho Time Series đa biến. Cung cấp feature importance trực tiếp và khoảng tin cậy.
- **Điểm yếu:** Chi phí huấn luyện đắt đỏ, đòi hỏi kiến thức cực sâu để tinh chỉnh (tuning).
- **Mục tiêu sử dụng:** Các cuộc thi định lượng cấp cao, dự đoán danh mục quỹ phòng hộ với nhiều biến phức tạp.

4 CODE - KAGGLE

4.1 Đôi điều về cuộc thi Hedge Fund

- **Mục đích của cuộc thi:** Cuộc thi không chỉ tìm kiếm một mô hình có độ chính xác cao nhất trên bảng xếp hạng (Leaderboard), mà còn nhằm tìm ra các giải pháp:
  - *Dự báo tuần tự (Sequential Prediction):* Mô phỏng chính xác cách thị trường vận hành. Tại thời điểm t, chỉ được biết dữ liệu quá khứ $[1 : t]$.
  - *Tính ổn định và thực tế:* Loại bỏ các mô hình "ăn may"hoặc dựa vào việc nhìn trộm tương lai (look-ahead bias).
  - *Tuyển dụng:* Đánh giá tư duy xử lý dữ liệu và khả năng thiết kế hệ thống thực chiến.
- **Phân tích Dataset:** Dữ liệu có cấu trúc phân cấp và trọng số phức tạp:
  - *Chiều thời gian:* Các quan sát sắp xếp theo trình tự t.

- *Cấu trúc thực thể:* Phân lớp qua các trường `code`, `sub-code`, và `sub_category`.
- *Trọng số (w_i):* Sai số trên các dòng có trọng số lớn ảnh hưởng cực nặng đến tổng điểm theo công thức Weighted Metric.
- *Biến mục tiêu (y):* Lợi suất (returns) hoặc biến động giá trị tài sản.

- **Các thuật toán lựa chọn ban đầu:**

4.2 Thuật toán có thể lựa chọn ban đầu

Dựa trên đặc thù của bộ dữ liệu (dữ liệu bảng với 84 feature, cấu trúc phân cấp, trọng số mẫu) và yêu cầu về khả năng tái lập trong 6 giờ, các thuật toán sau được đề xuất:

4.2.1 Mô hình hồi quy mạnh (Ưu tiên hàng đầu)

- **LightGBM / XGBoost:**
  - *Ưu điểm nổi bật:* Hỗ trợ trực tiếp `sample_weight` thông qua tham số huấn luyện, rất quan trọng vì cuộc thi cung cấp trọng số cho từng mẫu và hàm đánh giá là Weighted RMSE.
  - *Xử lý biến phân loại:* Có cơ chế xử lý biến categorical (`code`, `sub_code`, `sub_category`) hiệu quả mà không cần one-hot encoding tốn kém.
  - *Hiệu suất:* Hoạt động tốt với dữ liệu dạng bảng nhiều chiều (84 feature), ít nhạy cảm với nhiễu và có khả năng tự động chọn lọc đặc trưng quan trọng.
  - *Thời gian chạy:* Có thể điều chỉnh tham số để huấn luyện nhanh dưới 6 giờ.
- **CatBoost:**
  - *Điểm mạnh:* Xử lý biến phân loại vượt trội nhờ thuật toán target-based encoding đặc trưng, giảm thiểu overfitting khi có nhiều nhóm phân loại (code, sub-code).
  - *Tích hợp:* Hỗ trợ `sample_weight` và thường ít phải điều chỉnh tham số hơn.
- **Ridge / Lasso Regression:**
  - *Vai trò:* Là baseline ổn định để so sánh, đặc biệt khi dữ liệu có nhiễu (low signal-to-noise ratio).
  - *Chống overfitting:* Regularization L1/L2 giúp kiểm soát độ phức tạp khi có 84 feature, nhiều feature có thể không liên quan đến target.
  - *Hạn chế:* Không tự động xử lý biến phân loại, cần mã hóa (one-hot/target encoding) trước khi đưa vào mô hình.

4.2.2 Mô hình chuỗi thời gian thuần túy (Có thể kết hợp)

- **Mô hình cổ điển (ARIMA/SARIMA/ETS):**

    * *Phù hợp:* Xây dựng baseline nhanh cho từng chuỗi riêng lẻ (mỗi tổ hợp code-subcode-category-horizon).
    * *Hạn chế lớn:* Không thể kết hợp được 84 feature ngoại sinh, khó mở rộng cho hàng nghìn chuỗi khác nhau.
  - **Mô hình hiện đại dạng Transformer cho Time Series:**
    * *Temporal Fusion Transformer (TFT):* Được thiết kế riêng cho dự báo chuỗi thời gian, hỗ trợ biến ngoại sinh (84 feature), biến phân loại và xử lý nhiều horizon khác nhau.
    * *Tiềm năng:* Có thể học được mối quan hệ phức tạp giữa các feature và target theo thời gian.
    * *Thách thức:* Yêu cầu cấu hình dữ liệu dạng chuỗi (sequence), dễ gặp vấn đề về data leakage nếu không xử lý tuần tự cẩn thận; thời gian huấn luyện có thể vượt quá 6 giờ nếu không tối ưu.
  - **LSTM/GRU:**
    * *Ưu điểm:* Học được phụ thuộc dài hạn trong chuỗi thời gian.
    * *Nhược điểm:* Khó tích hợp 84 feature đa dạng, dễ bị overfitting nếu không đủ dữ liệu cho mỗi chuỗi.

4.2.3 Lưu ý đặc biệt từ dữ liệu quan sát được

Quan sát từ notebook `basic-example` cho thấy:

- **Số lượng feature lớn:** 84 feature từ `feature_a` đến `feature_ch` với nhiều giá trị null (được xử lý bằng `fill_null(0)`). Điều này ủng hộ các mô hình có khả năng tự động chọn lọc đặc trưng (XGBoost/LightGBM) hoặc có regularization (Ridge).
- **Trọng số mẫu (sample_weight):** Cột `weight` được cung cấp và sử dụng trong cả huấn luyện và đánh giá. LightGBM/XGBoost cho phép truyền trực tiếp tham số này qua `sample_weight` trong phương thức `fit()`, trong khi các mô hình Deep Learning cần xử lý thủ công qua hàm loss tùy chỉnh.
- **Cấu trúc phân cấp:** Các cột `code`, `sub_code`, `sub_category` là biến phân loại quan trọng. XGBoost/LightGBM/CatBoost xử lý tốt nhất với chi phí thấp nhất.
- **Yêu cầu thời gian:** Giới hạn 6 giờ chạy notebook khiến các mô hình Deep Learning quy mô lớn khó khả thi nếu không có tối ưu đặc biệt.

4.2.4 Đề xuất chiến lược thử nghiệm

1. **Bước 1 - Baseline:** Ridge Regression với feature đã chuẩn hóa (như basic-example) để có public score hợp lệ.
2. **Bước 2 - Gradient Boosting:** LightGBM với các tham số mặc định, tận dụng categorical feature, `sample_weight` và thử nghiệm với các đặc trưng thống kê theo nhóm (group-wise features).
3. **Bước 3 - Tối ưu hóa:** Tuning hyperparameters cho LightGBM/XGBoost, thử nghiệm feature engineering nâng cao.

4. **Bước 4 - Thử nghiệm nâng cao (nếu còn thời gian):** Triển khai TFT hoặc LSTM trên một mẫu nhỏ để đánh giá tiềm năng, nhưng cần đảm bảo pipeline tuần tự chống data leakage.

4.3 Thách thức

- **Nhiễu và tỷ lệ tín hiệu/nhiễu thấp (Low signal-to-noise ratio):** Dữ liệu tài chính vốn nổi tiếng với lượng nhiễu lớn, khiến việc tách bạch các mẫu hình thực sự (tín hiệu) khỏi biến động ngẫu nhiên (nhiễu) trở nên cực kỳ khó khăn. Mô hình dễ bị "học"cả nhiễu, dẫn đến overfitting.

- **Tính không dừng (Non-stationarity):** Quy trình sinh dữ liệu (underlying process) có thể thay đổi theo thời gian (ts_index). Điều này có nghĩa là các quy luật, mối tương quan hay phân phối thống kê trong quá khứ có thể không còn đúng trong tương lai. Một mô hình tốt không chỉ học từ quá khứ mà còn phải có khả năng thích ứng với những thay đổi này.

- **Thông tin đánh giá bị che giấu:** 75% dữ liệu kiểm tra (test data) cuối cùng được giữ kín hoàn toàn và chỉ được sử dụng để tính toán bảng xếp hạng cuối cùng (private leaderboard). Điều này tạo ra một thách thức lớn:
  - Tránh tối ưu quá mức (overfitting) lên 25% dữ liệu công khai (public leaderboard).
  - Đảm bảo mô hình có khả năng tổng quát hóa thực sự cao trên phần dữ liệu ẩn chưa từng thấy.

- **Cấu trúc phân cấp phức tạp:** Mối quan hệ chéo giữa các nhóm $code$, $sub-code$, và $sub-category$ tạo ra một cấu trúc dữ liệu phân cấp phức tạp. Việc mô hình hóa các mối quan hệ này một cách chính xác (ví dụ: điểm chung giữa các $code$ trong cùng $sub-category$, hay điểm riêng của cùng $sub-code$ ở các $code$ khác nhau) đòi hỏi các kỹ thuật xử lý đặc trưng tinh vi.

- **Yêu cầu nghiêm ngặt về "không rò rỉ dữ liệu tương lai"(No look-ahead bias):** Toàn bộ quá trình dự báo phải được thực hiện tuần tự. Dữ liệu tại thời điểm t chỉ được phép sử dụng thông tin từ quá khứ ($\leq t$). Sai sót trong bước này (ví dụ: sử dụng toàn bộ tập test để tính toán đặc trưng) sẽ dẫn đến mô hình "ảo", cho điểm số cao giả tạo trên public leaderboard nhưng bị loại ở vòng chung kết.

- **Nguồn lực tính toán và thời gian:** Mô hình cuối cùng phải được đóng gói trong một Jupyter Notebook có khả năng chạy lại toàn bộ quá trình (từ dữ liệu thô đến file dự báo) trong vòng dưới 6 giờ. Điều này đặt ra giới hạn về độ phức tạp của mô hình và quy trình tiền xử lý.

4.4 Mô Hình Lỗi - Phân tích từ "Incorrect Modelling Notebook

Notebook `example-of-an-incorrect-modelling` được ban tổ chức cung cấp nhằm minh họa các lỗi **data leakage** (rò rỉ dữ liệu tương lai) phổ biến mà thí sinh dễ mắc phải. Tất cả các lỗi này đều dẫn đến điểm số đánh giá cao một cách giả tạo trên tập train, nhưng mô hình sẽ thất bại trong môi trường thực tế và bị loại khỏi cuộc thi khi kiểm tra tính tái lập.

4.4.1 Lỗi 1: Chuẩn hóa target bằng thống kê toàn cục theo nhóm

Listing 1: Ví dụ chuẩn hóa target sử dụng thông tin tương lai

```
# Sai: Tính độ lệch chuẩn trên TOÀN BỘ nhóm (gồm cả tương lai)
df = df.sort('ts_index')
df = df.with_columns(
    (pl.col("y_target") /
     pl.col("y_target").std().over(["code", "sub_code", "
        sub_category", "horizon"]))
    .alias("y_target_norm")
)
df = df.drop_nulls(subset=["y_target_norm", "weight"])

X = df.select(cols).fill_null(0).to_numpy()
y = df.select("y_target_norm").to_numpy().ravel()
w = df.select("weight").to_numpy().ravel()

model = LinearRegression(fit_intercept=True)
model.fit(X, y, sample_weight=w)
y_pred = model.predict(X)
score = weighted_rmse_score(y, y_pred, w)
print("Example 1 Score:", score)  # Kết quả: 0.13078596396192552
```

Phân tích lỗi:

- Hàm `std().over(...)` tính độ lệch chuẩn của **tất cả** giá trị `y_target` trong cùng một nhóm (`code`, `sub_code`, `sub_category`, `horizon`).
- Điều này bao gồm cả những giá trị `y_target` ở các `ts_index` lớn hơn (tương lai) so với `ts_index` của hàng hiện tại.
- **Hậu quả:** Khi dự báo cho thời điểm t, mô hình đã "biết"trước độ phân tán (std) của toàn bộ chuỗi thời gian trong nhóm đó - một thông tin không thể có trong thực tế.

4.4.2 Lỗi 2: Tạo feature trung bình trên toàn bộ thời kỳ

Listing 2: Ví dụ tạo feature mean sử dụng dữ liệu tương lai

```
# Sai: Tính trung bình của feature trên TOÀN BỘ dữ liệu (gồm cả tương lai)
cols2 = ['feature_z','feature_aa','feature_ab',...,'feature_aw']
df = df.with_columns([
    pl.col(c).mean().alias(f"{c}_mean") for c in cols2
])

X = df.select(cols + [f"{c}_mean" for c in cols2]).fill_null(0).
   to_numpy()
y = df.select("y_target").to_numpy().ravel()
w = df.select("weight").to_numpy().ravel()

model = LinearRegression(fit_intercept=True)
```

```
model.fit(X, y, sample_weight=w)
y_pred = model.predict(X)
score = weighted_rmse_score(y, y_pred, w)
print("Example 2 Score:", score)  # Kết quả: 0.06338199908276958
```

Phân tích lỗi:

- `pl.col(c).mean()` tính **giá trị trung bình** của feature đó trên **tất cả** các hàng trong toàn bộ DataFrame, không phân biệt thời gian.
- Nó sử dụng cả dữ liệu từ tương lai (các `ts_index` lớn hơn) để tính toán cho quá khứ.
- **Hậu quả:** Mỗi hàng trong quá khứ được bổ sung thông tin "trung bình tương lai", tạo ra lợi thế không có thật.

4.4.3 Lỗi 3: Tạo feature trễ (lag) bằng dữ liệu tương lai

Listing 3: Ví dụ tạo lag feature với shift âm (lấy từ tương lai)

```
# Sai: Dịch chuyển feature lên trên (shift -50) để lấy giá trị từ TƯƠNG LAI
df = df.with_columns([
    pl.col(c).shift(-50).alias(f"{c}_lag50") for c in cols2
])

X = df.select(cols + [f"{c}_lag50" for c in cols2]).fill_null(0).
   to_numpy()
y = df.select("y_target").to_numpy().ravel()
w = df.select("weight").to_numpy().ravel()

model = LinearRegression(fit_intercept=True)
model.fit(X, y, sample_weight=w)
y_pred = model.predict(X)
score = weighted_rmse_score(y, y_pred, w)
print("Example 3 Score:", score)  # Kết quả: 0.06382524192951226
```

Phân tích lỗi:

- `shift(-50)` là một **dịch chuyển âm (negative shift)** - lấy giá trị của feature từ **50 hàng phía sau (trong tương lai)** và gán cho hàng hiện tại.
- Ví dụ: để dự báo tại `ts_index = 100`, mô hình được cung cấp feature là giá trị của một biến khác tại `ts_index = 150`.
- **Hậu quả:** Đây là dạng data leakage thô thiển nhất - sử dụng trực tiếp giá trị tương lai để dự báo quá khứ.

4.4.4 Tổng hợp so sánh

4.4.5 Bài học rút ra

- **Nguyên tắc vàng:** Không bao giờ để thông tin từ tương lai ($ts_index > t$) ảnh hưởng đến quá trình xây dựng đặc trưng hoặc mô hình tại thời điểm t.

Loại lỗi	Mô tả	Công thức/Hành vi sai	Hậu quả
Chuẩn hóa target	Dùng std toàn cục theo nhóm	`y_target / y_target.std().over(...)`	Biết trước độ phân tán tương lai của target
Feature mean toàn cục	Tính mean trên toàn bộ thời gian	`pl.col(c).mean()`	Biết trước giá trị trung bình tương lai của feature
Lag âm	Dịch chuyển về tương lai	`shift(-50)`	Trực tiếp sử dụng giá trị tương lai làm feature

Bảng 1: Tổng hợp các lỗi data leakage trong notebook mẫu

- **Các hàm cần thận trọng:** `.mean()`, `.std()`, `.over()`, `.shift()` với giá trị âm, hoặc bất kỳ phép tính thống kê nào trên toàn bộ DataFrame.
- **Quy tắc kiểm tra:** Nếu một phép biến đổi sử dụng dữ liệu từ các hàng có `ts_index` lớn hơn hàng hiện tại để tạo ra feature cho hàng hiện tại, đó là data leakage.
- **Hệ quả trong cuộc thi:** Các mô hình lỗi này có thể đạt điểm cao trên public leaderboard nhưng **chắc chắn bị loại** ở vòng kiểm tra tính tái lập và xác minh mã nguồn, do vi phạm quy tắc "No Data Leakage".

5 Phân tích Pipeline "Competition v5

5.1 Tổng quan kiến trúc

Pipeline này được thiết kế cho cuộc thi dự báo chuỗi thời gian với 5 trọng tâm chiến lược chính:

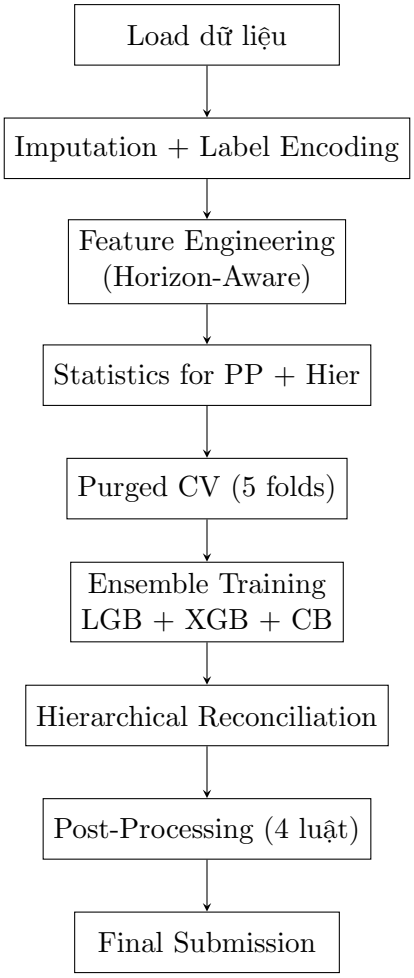
1. **Hàm đánh giá (Skill Score):** Tối ưu hóa trực tiếp theo Weighted RMSE
2. **Feature Engineering Horizon-Aware:** Tôn trọng tuyệt đối thứ tự thời gian
3. **Khai thác phân cấp (Hierarchy):** Tận dụng cấu trúc code-subcode-subcategory
4. **Validation chiến lược:** Purged Time-Series Expanding Window
5. **Ensemble đa dạng:** 5 seeds $\times$ 3 models = 15 predictions

5.2 Chi tiết các thành phần chính

5.2.1 Xử lý dữ liệu ban đầu (Cells 4-5)

- **Downcast to Float32:** Giảm dung lượng bộ nhớ, phù hợp với dữ liệu 1.4 triệu dòng
- **Imputation từ train:**

```
impute_vals = {}
for c in FEAT_COLS:
    med = train[c].drop_nulls().drop_nans().median()
    impute_vals[c] = float(med) if med is not None else 0.0
```



Hình 1: Pipeline tổng thể "Full Grand Strategy"@

- **Label Encoding an toàn:** Chỉ fit trên train, transform trên test

5.2.2 Feature Engineering Horizon-Aware (Cell 6)

Đây là điểm mạnh nhất của pipeline. Tất cả features được thiết kế để **không rò rỉ dữ liệu tương lai**:

A. Lag Features an toàn:

- `safe_lag_y`: Lag bằng đúng horizon → an toàn cho mọi horizon
- Các lag cụ thể (1,2,3,5,10,15,25): Model tự học lag nào phù hợp với horizon nào

B. Rolling Statistics:

```
pl.col('lag1_y').rolling_mean(5).over(GROUP_H).alias('roll5_mean'
    )
```

Quan trọng: Rolling được tính trên `lag1_y` (đã shift) chứ không phải target gốc → không nhìn thấy tương lai.

C. Hierarchical Features:

- `sc_r10_mean`: Trung bình 10 ngày của sub-category (mượt hơn code level)
- `dev_from_sc`: Độ lệch so với sub-category → phát hiện "divergence"
- `share_of_sc`: Tỷ trọng của code trong sub-category

D. Target Encoding an toàn:

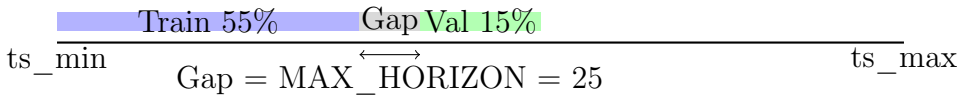
```
pl.col('lag1_y').rolling_mean(28).over(GROUP_SB).alias('te_sb_28'
    )
```

Sử dụng rolling 28 ngày thay vì mean toàn bộ lịch sử để tránh look-ahead.

E. Noise Detection Features:

- `zero_r30`: Tỷ lệ zero trong 30 ngày (Luật 1)
- `cv_30`: Coefficient of Variation (Luật 2)

5.2.3 Purged Time-Series Cross-Validation (Cell 10)



Hình 2: Cơ chế Purged Gap trong Cross-Validation

- **5 folds với expanding window:** Mô phỏng chính xác bối cảnh thi đấu
- **Purge gap = MAX_HORIZON (25):** Ngăn model "ăn gian"thông tin từ các horizon dài
- **Fold cuối cùng Public LB:** Fold 5 (85% train, 15% val) gần nhất với cấu trúc test

5.2.4 Ensemble Training (Cell 13)

Model	Số rounds	Device	Đặc điểm
LightGBM	BEST_LGB (median từ CV)	CUDA/CPU	max_bin=127 trong Dataset
XGBoost	BEST_LGB × 0.9	CUDA/CPU	tree_method='hist'
CatBoost	BEST_LGB × 0.9	GPU/CPU	task_type='GPU'

Bảng 2: Cấu hình 3 mô hình cốt lõi

Seed Blending: 5 seeds × 3 models = 15 predictions

- Mô hình A đoán hơi cao + mô hình B đoán hơi thấp → bù trừ nhau
- Luật số lớn (Law of Large Numbers) triệt tiêu sai số ngẫu nhiên

5.2.5 Hierarchical Reconciliation (Cell 14)

Kỹ thuật Top-Down Reconciliation:

1. Tính tổng predictions của các code trong cùng sub-category
2. Lấy "anchor"từ historical mean: `sc_total_mean × số lượng items`
3. Scale: `p_rec = p_raw × (anchor / sum_pred)`
4. Blend với weight `HIER_RECONCILE_W = 0.4`

Công thức:

$$p_{\text{final}} = w \cdot p_{\text{raw}} \cdot \frac{\text{anchor}}{\sum p_{\text{raw}}} + (1 - w) \cdot p_{\text{raw}}$$

5.2.6 Post-Processing: 4 Luật Chống Nhiễu (Cell 15)

Luật	**Điều kiện**	**Hành động**
1	zero_ratio > 80%	prediction = 0.001
2	CV > 3 & biến động lớn	cap tại mean ± 10% std
3	|p - median| > 3	kéo về median ± 2.5
4	horizon 10	blend về mean (30%-70%)

Bảng 3: 4 luật post-processing từ tài liệu

Các luật này phản ánh triết lý **"dự đoán bảo thủ"**: Khi không chắc chắn, hãy ép về giá trị an toàn (mean, median, 0).

5.3 So sánh với Basic Pipeline

Tiêu chí	**Basic Pipeline**	**Full Grand Strategy**
Feature Engineering	Chỉ dùng feature gốc	Lag, rolling, hierarchical, target encoding, noise features
Validation	Train/Test đơn giản	Purged 5-fold Expanding Window
Mô hình	Linear Regression	Ensemble: LGB + XGB + CB × 5 seeds
Xử lý horizon	Không phân biệt	Horizon-aware features, safe_lag
Post-processing	Không có	4 luật chống nhiễu + Hierarchical Reconciliation

Bảng 4: So sánh Basic vs Full Grand Strategy

5.4 Điểm mạnh và điểm yếu

5.4.1 Điểm mạnh

- **No Leakage tuyệt đối:** Mọi feature đều được thiết kế để không nhìn thấy tương lai
- **Robust với noise:** 4 luật post-processing + hierarchical reconciliation
- **Stability cao:** 15 predictions ensemble giảm variance
- **Memory efficient:** Polars + Float32 downcast + batch prediction

5.4.2 Điểm yếu

- **Độ phức tạp cao:** Khó debug, khó maintain
- **Time-consuming:** 15 models training có thể gần 6 giờ
- **Over-engineering risk:** Có thể một số features không thực sự helpful

5.5 Kết luận

Pipeline "Full Grand Strategy"là một implementation xuất sắc của các nguyên lý từ tài liệu:

1. **Tôn trọng thời gian:** Mọi feature đều được thiết kế horizon-aware
2. **Khai thác cấu trúc:** Hierarchical features + Reconciliation
3. **Chống nhiễu:** 4 luật post-processing dựa trên statistical signals
4. **Ổn định:** Ensemble + Seed Blending

Điểm mạnh nhất là sự **kết hợp chặt chẽ giữa lý thuyết (các trọng tâm từ tài liệu) và thực hành (code implementation cụ thể)**.